COMP 3700A: Contributing to Open Source

More adventures with `git`

Charlotte Curtis

October 19, 2026

Housekeeping

- Welcome back! Where were we? Where are we going?
    - Midterm in class next Wednesday
    - Up to and including this week (More git stuff, maybe funding models)
    - Conceptual understanding (e.g. "what does it mean to rebase", not "what is the command to rebase onto upstream")
- Do we have notetakers for today's issues?
- How goes part 2?

> ***Important:*** *If my instructions don't make sense in the context of your chosen project, make a note of this and describe what does make sense*

Today's topics

Merge conflicts:

- Merge strategies
- Resolving merge conflicts
- Undoing changes

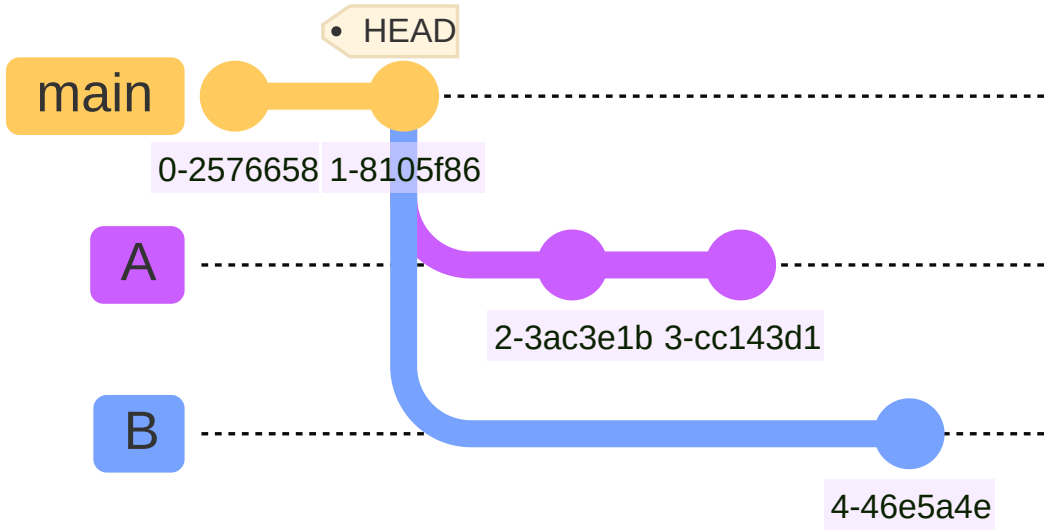
Interactions with upstream:

- Definitions
- Cherry-picking changes
- Git `bisect`
- Submodules and external patches

Resources: git documentation, GitKit book

> *Some of this may spill over to Wednesday's class*

The problem




```python
for i in range(10): # main
    do_something_with(i)
```

```python
for n in range(10): # A
    do_something_with(n)

print("Done")
```

```python
print("Starting loop...")
for i in range(20): # B
    do_something_with(i)
```

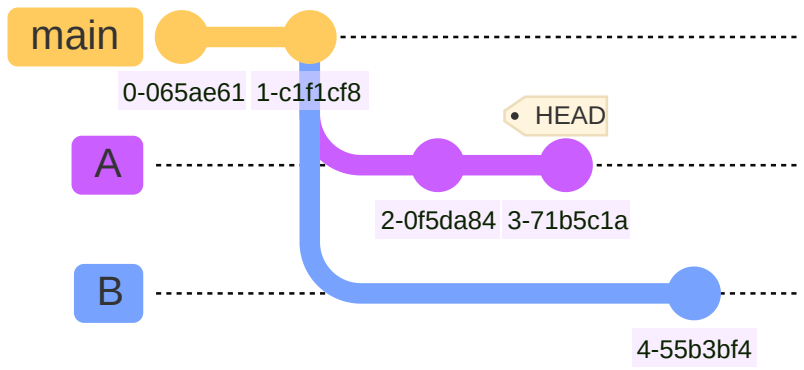
The solution: 3-way merge

- `git merge` uses one of several strategies (default: **ort**)
- At a high level, the algorithm is as follows:
    i. Find the best common ancestor
    ii. Find the changes in each branch
    iii. Apply non-conflicting changes, fast-forwarding where possible
    iv. Stop and prompt for interaction in case of conflict

Fast-forward vs. merge

Fast-forward

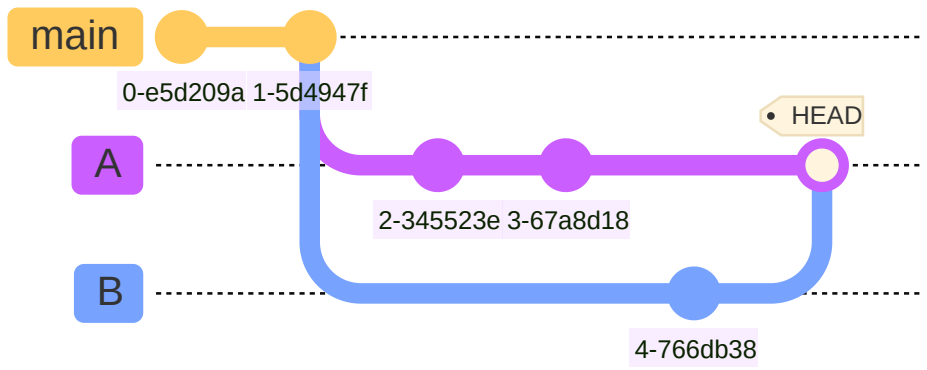
`git merge A`



- `HEAD` pointer is simply moved
- No new commit is created

Merge

`git merge B`



- Divergent history requires a patch
- A new merge commit is created

Conflict? Don't panic!

- You can always `git merge --abort` to go back to how things were
- You can resolve the conflict manually:

```
<<<<<<< HEAD
for n in range(10):
    do_something_with(n)
=======
print("Starting loop...")
for i in range(20):
    do_something_with(i)
>>>>>>> B
```

- then `git add` and `git merge --continue`

`reset`, `restore`, `revert`???

I want to...	Use
Change the last commit	`git commit --amend`
Remove a file from the staging area	`git restore --staged <file>`
Discard uncommitted changes	`git restore <file>`
Rewind log to a previous commit	`git reset <commit>` [`--hard`]
Inspect the code a previous commit	`git checkout <commit>`
Make a new commit undoing a previous one	`git revert <commit>`

Things in **bold** are dangerous operations resulting in permanent changes!



When have you had trouble with git? What was the solution?

Post your response in the Homework Discussion

Your `reflog` has more info

- Running `git reflog` shows a record of all the actions you've taken *locally*
- You can use this to recover from mistakes
- Example scenario:

> *I used `git reset` to go back to a previous commit, then realized I went back to the wrong one. How do I get back?*

- Once you push changes, recovering from mistakes gets trickier

Interactions with upstream

The many meanings of "upstream"

"Upstream" can refer to:

- The canonical source repo that you forked and connected to with:

  ```
  git remote add upstream <url>
  ```

- A dependency for the current project:

  "This bug seems to be an issue in the upstream GUI toolkit"

- "Upstream" as a verb, shorthand for "contribute changes to upstream":

  "Go ahead and upstream your patch"

Getting changes from upstream

- We've already talked about rebasing onto `upstream/main`
- What if you want a single commit from another branch?
  `git cherry-pick <commit>`
- Also useful for splitting your own work into multiple commits
- As with `merge`, a conflict may occur, which you can resolve and then `git cherry-pick --continue` or choose `git cherry-pick --abort`

Upstream conventions

- Commit messages may follow certain rules, e.g.:
  - conventional commits, particularly popular with AI-friendly projects
  - .gitmessage file guidelines (e.g. Inkscape)
  - Why is this important? Here's an example
- Pull requests may follow certain rules, e.g.:
  - Squashing to a single commit
  - Referencing the associated issue
  - Just like issues, a template may exist (e.g. Pattern Projector)

Chasing down trouble

- Some bug reports are **regressions** (and may be labelled as such)
- It's usually necessary to find the change that caused the regression
- `git bisect` can help with this:
    i. `git bisect start`
    ii. `git bisect bad`
    iii. `git bisect good <last-good-commit>`
    iv. Compile, test, and repeat steps 2-3 until 0 steps left

Managing dependencies: Submodules

- Sometimes, dependencies can't be managed with package managers
- Some projects are split into multiple repos to keep things separate
- Git submodules are a (sometimes confusing) solution
- Example: Inkscape (again)
- Useful commands:

```
git clone --recurse-submodules
git submodule update --init --recursive
```

> *Submodules are tracked separately from the main repo; never do git operations from inside a submodule directory!*

Managing dependencies: custom patches

- Sometimes source code from upstream dependencies needs modification, but you don't want to upstream those changes
- It might make sense to maintain a `.patch` file
- Patches are applied while building dependencies for the current project
- Example: Brave browser

Reporting issues upstream

- Chasing down a problem can involve multiple projects
- This is a great way to expand your FOSS exposure!
- Here's an example that seemed like an Inkscape bug, but ended up being a LuaTex/CapyPDF interaction issue
- When reporting upstream, it is helpful to link to the downstream project issue
- GitHub and GitLab support cross-project linking, but not cross-platform

> *Wherever possible, link to the associated item!*

Coming up next

- Discussion about funding models
- Lab exercise: git merging
- Review session
- Midterm